

LAPORAN TUGAS

**WEB SERVER
(TIF 330805)**

MINGGU KE 3



ACARA 5

DIBUAT OLEH:

NAMA: CINTYA AZWA SAFRINA R.H

NIM:E41250631

GOL:A

**PROGAM STUDY DILUAR KAMPUS UTAMA(PSDKU) POLITEHNIK NEGRI JEMBR
TEHNIK INFORMATIKA KAB.NGANJUK JL. JENDRAL GATOT SUBROTO NO
106,KAUMAN,KECAMATAN NGANJUK,KABUPATEN NGANJUK JURUSAN
TEKNOLOGI INFORMASI POLITEHNIK NEGRI JEMBER TAHUN 2026**

PENDAHULUAN

Dalam pengembangan aplikasi web tradisional, setiap halaman biasanya diakses langsung melalui nama file, misalnya `http://localhost/aplikasi/app/Controllers/MahasiswaController.php?action=index`. URL semacam ini memiliki beberapa kelemahan: sulit diingat, terlihat tidak profesional, serta membuka struktur folder aplikasi ke publik sehingga berpotensi menimbulkan celah keamanan.

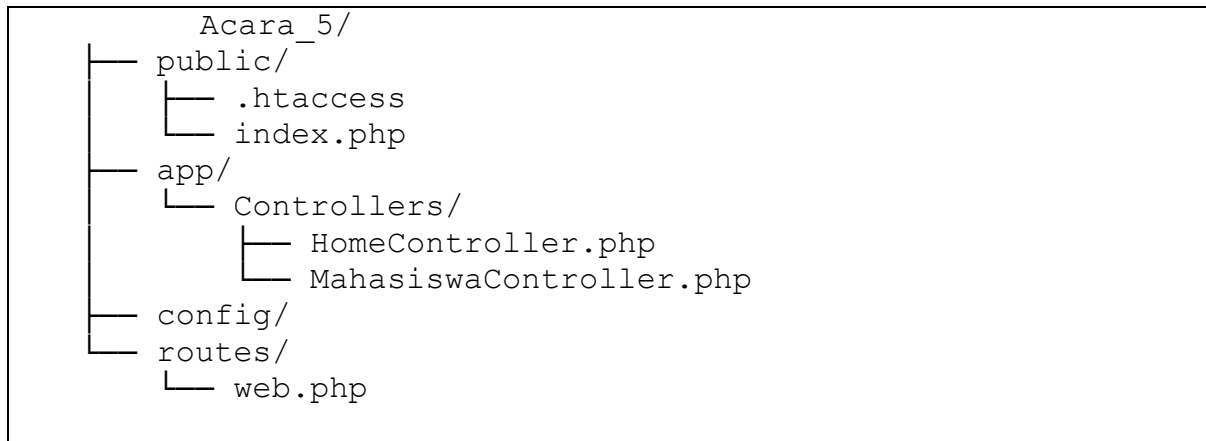
Untuk mengatasi hal tersebut, digunakan konsep **routing**, yaitu proses memetakan sebuah URL request ke kode program yang sesuai (biasanya method tertentu di dalam Controller). Router berperan seperti "petugas" yang menerima permintaan dan mengarahkannya ke tujuan yang tepat berdasarkan URL dan metode HTTP (GET atau POST) yang digunakan.

Agar seluruh request dapat diarahkan ke satu titik masuk (*entry point*), digunakan file **.htaccess** yang memanfaatkan modul `mod_rewrite` pada Apache. Dengan konfigurasi ini, setiap request yang tidak mengarah ke file atau folder nyata akan diteruskan ke `public/index.php`, yang selanjutnya bertindak sebagai **front controller** — satu-satunya pintu masuk yang bertugas membaca URL, mencocokkannya dengan daftar rute (*routes*) yang telah didefinisikan, kemudian memanggil Controller dan method yang sesuai.

Konsep ini juga melibatkan pemahaman terhadap **superglobal** PHP seperti `$_GET`, `$_POST`, dan `$_SERVER`, yang masing-masing berperan dalam mengambil data dari query string, data form, maupun informasi request itu sendiri (seperti URI dan method yang digunakan).

IMPLEMENTASI

1. Membuat struktur folder project baru bernama Acara_5 di dalam htdocs, dengan struktur:



2. Membuat file public/.htaccess berisi aturan RewriteEngine agar seluruh request yang bukan file/folder nyata diteruskan ke index.php.
3. Mengaktifkan AllowOverride All pada konfigurasi Apache (httpd.conf) agar file .htaccess dapat berfungsi, kemudian me-restart Apache.
4. Membuat file routes/web.php berisi array pemetaan URL dan method HTTP (GET/POST) ke pasangan Controller dan method tujuan.
5. Membuat HomeController.php dan MahasiswaController.php sebagai Controller sederhana yang masing-masing method-nya menampilkan output echo sebagai bukti route berhasil dipanggil.
6. Membuat public/index.php sebagai front controller yang membaca URI dari \$_SERVER['REQUEST_URI'], mencocokkannya dengan array routing, lalu memanggil Controller yang sesuai atau menampilkan 404 jika tidak ditemukan.
7. Menambahkan dukungan parameter URL dinamis (/mahasiswa/{id}) menggunakan preg_match dengan pola regex, sebagai penyelesaian Tugas Mandiri.
8. Menguji seluruh route: /, /mahasiswa, /mahasiswa/create, URL yang tidak terdaftar, dan /mahasiswa/5.

Struktur Kode

Routing sederhana (routes/web.php)

```
$routes = [
    'GET' => [
        '/' => ['HomeController', 'index'],
        '/mahasiswa' => ['MahasiswaController', 'index'],
        '/mahasiswa/create' => ['MahasiswaController', 'create'],
    ],
    'POST' => [
        '/mahasiswa' => ['MahasiswaController', 'store'],
    ],
];
```

Rewrite rule (public/.htaccess)

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]
```

Front controller dengan dukungan parameter URL (public/index.php)

```
$uri = urldecode(parse_url($_SERVER['REQUEST_URI'],
    PHP_URL_PATH));
$base = '/web server/minggu 3/Acara_5/public';
if (str_starts_with($uri, $base)) {
    $uri = substr($uri, strlen($base)) ?: '/';
}

$method = $_SERVER['REQUEST_METHOD'];

// Tugas Mandiri: dukungan parameter URL, misal /mahasiswa/5
if ($method === 'GET' && preg_match('#^/mahasiswa/(\d+)$#',
    $uri, $matches)) {
    $id = (int) $matches[1];
    $controller = new MahasiswaController();
    $controller->show($id);
    exit;
}

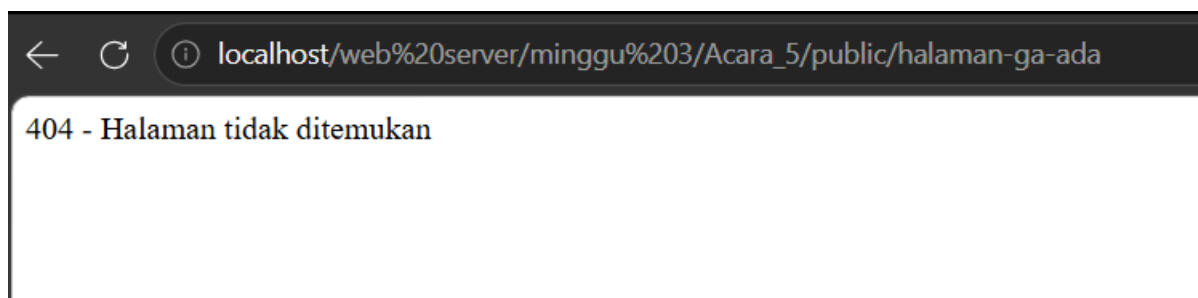
if (isset($routes[$method][$uri])) {
    [$controllerName, $action] = $routes[$method][$uri];
    $controllerClass = "App\\Controllers\\{$controllerName}";
    $controller = new $controllerClass();
    $controller->$action();
} else {
    http_response_code(404);
    echo "404 - Halaman tidak ditemukan";
}
```

Method tambahan untuk parameter dinamis (app/Controllers/MahasiswaController.php)

```
public function show($id)
{
    echo "<h1>Detail Mahasiswa</h1>";
    echo "<p>Menampilkan data mahasiswa dengan ID: <strong>{$id}</strong></p>";
}
```

Hasil Tampilan





Analisis

Hasil pengujian menunjukkan bahwa seluruh URL yang diakses tanpa ekstensi .php — seperti /, /mahasiswa, dan /mahasiswa/create — berhasil diarahkan ke method Controller yang sesuai. Hal ini membuktikan bahwa file .htaccess berhasil meneruskan seluruh request ke public/index.php, dan proses pencocokan URI terhadap array \$routes pada front controller berjalan dengan benar.

Ketika URL yang diminta tidak terdaftar dalam array routing (misalnya /halaman-ga-ada), aplikasi menampilkan pesan "404 - Halaman tidak ditemukan" yang berasal dari kode PHP sendiri (menggunakan `http_response_code(404)`), bukan pesan error bawaan Apache. Hal ini menunjukkan bahwa penanganan URL yang tidak dikenal telah diimplementasikan dengan baik pada level aplikasi.

Pada penyelesaian Tugas Mandiri, dukungan parameter URL dinamis diimplementasikan menggunakan fungsi `preg_match()` dengan pola regex `^/mahasiswa/(\d+)$`, yang mendeteksi apakah URI berbentuk /mahasiswa/ diikuti oleh angka. Jika pola cocok, nilai angka tersebut diekstrak dan dikirimkan sebagai parameter \$id ke method `show($id)` pada MahasiswaController. Pengujian dengan URL /mahasiswa/5 berhasil menampilkan ID 5 secara dinamis, membuktikan bahwa parameter benar-benar diambil dari URL, bukan nilai statis yang di-hardcode dalam program.

Secara keseluruhan, praktikum ini menunjukkan bagaimana konsep routing, .htaccess, dan front controller bekerja sama untuk menghasilkan URL yang bersih, profesional, dan mudah dikembangkan, sekaligus menjadi fondasi penting sebelum penerapan fitur-fitur backend yang lebih kompleks seperti pengambilan data form melalui `$_POST` dan penanganan session/cookie pada pertemuan berikutnya.

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Routing memungkinkan pemetaan URL ke Controller dan method tertentu, sehingga struktur URL aplikasi dapat dibuat bersih dan tidak menampilkan struktur folder secara langsung.
2. File .htaccess dengan modul mod_rewrite memungkinkan seluruh request diarahkan ke satu titik masuk (index.php), yang merupakan syarat utama penerapan pola front controller. Konfigurasi AllowOverride All pada Apache wajib diaktifkan agar .htaccess dapat berfungsi.
3. Superglobal \$_SERVER['REQUEST_URI'] dan \$_SERVER['REQUEST_METHOD'] berperan penting dalam menentukan URL dan metode HTTP yang digunakan, yang menjadi dasar pencocokan pada array routing.
4. Penanganan kondisi URL yang tidak terdaftar (404) penting untuk memastikan aplikasi tetap memberikan respons yang jelas kepada pengguna, bukan menampilkan error bawaan server.
5. Dukungan parameter URL dinamis dapat diimplementasikan menggunakan regular expression (preg_match), memungkinkan satu route menangani berbagai nilai parameter (seperti ID) tanpa perlu mendaftarkan setiap kemungkinan URL secara eksplisit.
6. Pemahaman terhadap routing dan .htaccess menjadi fondasi penting menuju implementasi fitur backend yang lebih kompleks, seperti penanganan form dengan GET/POST, validasi input, serta session dan cookie.